

Digital Communications

Work to be delivered

2023

Instructions

The assignments must fully comply with the instructions, otherwise they will not be graded. The relevant instructions are formulated as follows:

The execution and writing of the work is **individual**, as is the delivery. Collaborations are not permitted.

All answers will be collected in ONE doc file. This file will contain:

• The student's full details

• All the **pronunciations** of the exercises in the order they have been given. Below the pronunciation of the exercise to be answered, the solution will follow. In the event that an exercise is not to be answered, there will be only its pronunciation.

• The solution to each exercise will **necessarily include:**

- o The code (in Matlab, Octave ... software) that solves - answers the question.
- o Explanation of the commands used
- o Commenting on the solution
- o The m files you will create (copy –paste)
- o Commenting on the results
- o Theoretical solution (where necessary)
- o Figures, graphs related to the solution and results
of the Exercise

This file will be delivered - without any compression - to a special directory in eclass, at the end of the laboratory classes.

The assignments will be processed for plagiarism checking by academic paper detection, copying and plagiarism software. In case the same answer is found in 2 or more assignments (even in one of the questions), all these assignments will be nullified.

Some assignments may also be examined orally.

Good luck!

Digital Communications

Laboratory exercises to be delivered

1) **Sinusoidal signal.** Create a function that generates a sinusoidal signal. The arguments of the function should be the frequency of the signal, its sampling frequency, its amplitude, its phase and its time duration. The outputs of the function should be the sinusoidal signal and the time axis on which it was defined, e.g. $[x,t]=\text{mine_sin}(f_0,fs,A,ph,T1)$.

2) **Spectrum function.** Create a function that calculates the amplitude spectrum of a sinusoidal signal using the *fft()* function. The arguments of the function are the signal and its period.

3) **Sine wave spectrum.** Using the functions of questions 1) and 2), create a sine wave with a frequency of 10 Hz and a sampling frequency of 1KHz, the amplitude and phase of the sine wave being 5 Volt and 00 respectively and the duration 0.5 sec. Graph the sine, find its spectrum, and graph the amplitude spectrum. Compare the amplitude spectrum depicted in the graph with the theoretical expression for the amplitude spectrum of the sine.

4) **Speech signal spectrum.** Download and save the speech signal 3WORDS.WAV found at <https://booksite.elsevier.com/9780080993881/> (data directory). Read the signal with the audioread function and save it in a vector. Listen to the signal using the sound function. Plot the waveform of the signal as a function of time, and represent the amplitude spectrum of the signal.

5) **Spectrum of a noisy signal.** Create an information signal consisting of the superposition of cosine signals from 500Hz to 3000Hz, with a step of 100Hz.

Create the time axis by defining the elementary time change, dt , based on the fastest component and the observation duration based on the slowest component. Create the graphical representation of the signal.

You can also listen to the signal with `sound(x)`. Add 10 dB white additive Gaussian noise to the signal using `awgn()`. Also create a 2nd noisy signal with 5dB noise added. The corresponding graphs should be made in new graphic windows.

(Also listen to the two beeps).

To represent the amplitude spectrum of all signals.

6) **Melody spectrum.** Given the following Matlab program. Listen to the signal (`xx(t)`), to which musical notes do the frequencies of which it is composed correspond and to which known melody does the signal correspond? Construct the spectrum of the melody signal.

```
fs=6000;ts=1/
fs;t=0:ts:1;
fr=[1567 1760 1975 1046];
x=zeros(length(fr), length(t));
for i=1:length(fr)
    x(i,:)=cos(2*pi*fr(i)*t);
end
xx=[x(1,:) x(2,:) x(3,:) x(4,:)];
plot([0:ts:4+3*ts],xx)
sound(xx,fs)
```

7) **DSB modulation.** Create an information signal consisting of the superposition of cosine signals from 500Hz with a step of 100Hz to 3000Hz.

Carrier frequency 100KHz. Perform DSB modulation. Then use the function that implements an ideal (quadratic) simplified bandpass filter:

function `y = bpfilt(signal, f1, f2,fs)`

which is listed below and its arguments are, `signal` : the signal we want to filter, `f1`, `f2` : the lower and upper frequency limits of the bandpass filter and `fs` : the sampling frequency.

Using the function, you produce a) a USB – upper sideband and b) a LSB – lower sideband modulated signal.

Visualize the amplitude spectra of all signals.

```
%%%%%%%%%
```

```
function y = bpfilt(signal, f1, f2,fs)
```

```
%% Bandpass filtering
```

```
%
```

```
% Syntax:
```


8) **Aliasing phenomenon.** Representation of the aliasing phenomenon due to subsampling.

a) Create a 1KHz sinusoidal signal, 2 sec long. Sample the signal at 20KHz and then at 1.5KHz. Plot both signals on the same graph.

b) Listen to the two signals with the function `soundsc(x,fs)`. What do you observe?

c) Create a sine wave that will produce the same sound as the sine wave sampled at 1.5KHz.

9) **AM modulation.** Create an AM signal $x_1(t)$ with the following parameters: $A_c = 1$ Volt, $\gamma_m = 0.5$ Volt, $f_m = 2$ Hz, $f_c = 100$ Hz, when the baseband signal is $s(t) = \gamma_m \sin(2\pi f_m t)$. Assume that $t = [0:0.001:1]$.

Plot the modulated signal with `plot(t,x1)`. Also plot the envelope of the signal.

Modulate the same signal $s(t)$ with the `ammod()` function of Matlab's communication toolbox and name the signal $x_2(t)$.

What type of configuration does `ammod` implement?

Syntax `ammod`: $y = \text{ammod}(s, f_c, f_s)$

y : modulated signal

s : information signal

f_c : carrier frequency

f_s : sampling frequency - set $f_s = 400$

10) **Quantization.** Create a sampled noise signal, normally distributed (use `randn`) with a time axis $t = 0:0.001:5$. Quantize it a) in 8 and b) in 16 quantization levels. Plot the original signal, the quantized signal, the quantization level limits, the quantization bands and the quantization error in the same graphic window.

11) **Pulse code modulation.** Create a program that will PCM modulate a sine wave signal with an amplitude of 4V and a frequency of 10 Hz. The quantizer will use a 4-bit code word.

Draw the original signal, the quantized signal, and the quantization error.

12) **AWGN.** A white noise signal consists of a set of independent and identically distributed (iid) random variables. In the discrete case, the white noise signal consists of a sequence of samples that

are independent and are generated from the same probability distribution (e.g. uniform or Gaussian).

White Gaussian noise can be generated in Matlab with the *randn* function.

For example, the following commands generate White Gaussian noise (length 30 samples) with zero mean and standard deviation $\sigma=1$.

```
mu=0;sigma=1;  
noise= sigma *randn(1,30)+mu
```

The Power Spectral Density function (PSD) shows the amount of power contained per unit of spectrum.

The power spectral density of a random process can be calculated from the Fourier transform of its autocorrelation function.

It is assumed that its value is constant for the entire frequency spectrum and equal to the variation of the noise signal power.

A simple estimator of the psd is the periodogram. It consists of taking the DTFT of the signal samples and then squaring the magnitude of the result. The Matlab function periodogram can be used to calculate and plot the periodogram.

Generate a white Gaussian noise signal of length 100000 samples using randn and plot it. Let's assume that the Gaussian pdf has mean 0 and standard deviation $\sigma=2$ (variance $\sigma^2=4$)

In a new graph, create the histogram of the signal (consider 100 bins) and determine the characteristic shape of the Gaussian pdf.

Generate a second Gaussian white noise signal of length 100000 samples using randn and plot it. Let's assume that the Gaussian pdf has mean 2 and standard deviation $\sigma=1$ (variance $\sigma^2=1$)

Create the histogram of the signal and plot it on the same graph as the previous histogram.

13) AWGN power spectral density. (psd, power spectral density). Create a random process of white Gaussian noise consisting of 1000 random sequences of 512 samples each following a Gaussian distribution (e.g. in a matrix: v 1000x512).

Each sequence will have a mean of zero and a standard deviation of $\sigma=2$.

For each sequence use the following code to find the psd

```
V(i,:) = abs(fft(v(i,:))).^2/512; % periodogram
```

Find the mean of the psd and plot the result. Verify that the value of psd is constant and equal to $\frac{1}{2}$.

14) **Sinc function.** Generate the signals given by the following relationship

$$m1(t) = \text{sinc}(2\pi f_1 t)$$

$$m2(t) = \text{sinc}(4\pi f_1 t)$$

Where $f_1 = 100\text{Hz}$.

Make a graphic representation of the signals from -0.05 to 0.05 sec with a sampling step $\Delta t = 0.0001$. Also represent the x-axis on

the two graphs. At what points do the zeros of the signals occur and why? Make a representation of the amplitude spectrum of the signals but find the number of points of the fft (and the f-axis) by the command:

$$Nf = 2^{\lceil \log_2(N) \rceil};$$

Where N is the number of points on the t-axis. Why is this command used?

What is the theoretical spectrum of the signals $m1(t)$ and $m2(t)$? Compare the width of the two spectra and explain.

15) **DSB modulation. Spectrum.** To create an information signal given by the following relationship

$$m(t) = \text{sinc}(2\pi f_1 t)$$

Where $f_1 = 100\text{Hz}$.

The spectrum of the signal should be displayed but the number of points of the fft (and the f-axis) should be found by the command:

$$Nf = 2^{\lceil \log_2(N) \rceil};$$

Where N is the number of points on the t-axis.

Modulate the signal using DSB with a carrier frequency of 600Hz. Graphically display the modulated signal and its spectrum.

16) **DSB demodulation.** a) Create a cosine signal with amplitude $\hat{y}_m = 0.5$ Volt, frequency $f_a = 1\text{KHz}$. The time axis should be defined from 0 to 5 periods of the signal. Create a graphical window and divide it into 3 subwindows. In the 1st subwindow, plot $x(t)$ versus t.

b) Create a cosine signal with amplitude $y_c=2.5$ Volt, frequency f_c ten times the frequency of the information signal. The time axis remains the same.

Display the graph of $y(t)$ versus t in the 2nd subwindow.

c) Modulate the carrier signal, $y(t)$, from the information signal, $x(t)$, using DSB.

The modulated signal, $z(t)$, is represented in terms of t , in the 3rd subwindow.

d) Perform coherent demodulation of the signal, using a local oscillator, coherent with the transmitter carrier signal and then pass the signal through a low-pass filter *butterworth_filter*(*in*, *dt*, *order*, *fcut*, *fcenter*), using the corresponding file (select the appropriate parameters).

In a new graphic window, simultaneously represent (with the window frozen) the original information signal and the demodulated one (after the filter). Choose different colors for the two graphics.

17) **DSB demodulation with non-ideal local oscillator.** Repeat the process of question (d) of exercise 16, for a non-ideal local oscillator with some small phase difference from the carrier signal (e.g. 0.1π). In a new graphic window, simultaneously represent the original information signal and the demodulated one (after the filter).

Repeat the process for a phase difference such that there is a total loss of the demodulated signal. In a new graphic window, simultaneously represent the original information signal and the demodulated one (after the filter).

18) **Shannon Hartley Theorem.** Let us consider the Shannon Hartley theorem:

$$C = \frac{1}{2} \log_2 \left(1 + \frac{P}{N} \right) \quad /$$

If instead of the average power N of the noise we consider the power spectral density $N_0/2$ then the above relationship takes the form:

$$C = \frac{1}{2} \log_2 \left(1 + \frac{P}{N_0 B} \right) \quad /$$

When the S/N ratio tends to infinity, then the capacity of channel C also tends to infinity. Correspondingly, when the bandwidth of channel B tends to infinity, the capacity of channel C tends to a specific value:

$$\lim_{B \rightarrow \infty} C = 1.44 \frac{P}{N_0} \quad /$$

Write a program to calculate and represent the channel capacity C with a frequency bandwidth $B=3000\text{Hz}$ as a function of the S/N_0 ratio (give values for the S/N_0 ratio from -20 to 30dB). For value compression, use the `semilogx` command instead of `plot`.

19) **Channel capacity.** Write a program to calculate and represent the capacity of channel C with an S/N_0 ratio equal to 25dB as a function of the frequency bandwidth of channel B (values from 1 to 100000). For value compression, use the `semilogx` command instead of `plot`.

Is the value to which the capacity tends the expected value based on Theory?

20) **Huffman coding.** Create the message sequence:

'my name is <insert name here>'

Store the message in a variable named `myname`.

Create the source alphabet and the vector of symbol occurrence probabilities. Create the Huffman codebook. Perform Huffman encoding of the message. Calculate the source entropy, the code efficiency and its redundancy.

21) **Binary PSK Modulation.** Let the message sequence be $m=[1\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1]$. Let each pulse – bit have a duration of $T=1\text{ms}$ and the sampling interval is $dt=10^{-7}\text{ s}$. Let the carrier have a frequency of $f_c=5000\text{Hz}$. Modulate the message using binary PSK and then plot the spectrum of the information signal and the modulated one.

22) **QAM modulation.** Run the following Matlab code to generate and graphically display 16QAM modulated data. Explain in detail what each command does.

Write the corresponding code for a) 16QAM and $\text{SNR}=5\text{dB}$ b) 4 QAM and 20dB .
Use the `scatterplot()` function alternatively to produce signal constellations.

23) **Text Entropy.** Write code to calculate the Entropy of an English text

Hint: you can identify the different letters that appear in the text and then calculate the probability of their appearance (frequency of appearance of the character / total characters in the text). Then calculate the entropy based on the definition. (For simplicity, you can consider that the text consists only of letters and spaces. You can also consider a short text and store it in a variable e.g. `text='This is a short text';`) What are the probabilities of the appearance of the characters in the text?

24) **Source entropy.** Given a DMS source with transmitted symbol probabilities:

0.2, 0.05, 0.03, 0.1, 0.3, 0.02, 0.22, 0.08.

Apply Huffman encoding to the source and then generate a sequence of 200 source symbols. The sequence should be Huffman encoded.

The program should display for each symbol of the source ($x_1, x_2, \dots$) the probability of its appearance and the code word assigned to it, the printout will be as follows:

x1 probability: 0.3 codeword :--->
0 0

x2 probability : 0.25 codeword :--->
0 1

.....

Then, the source entropy and the average codeword length are calculated. The efficiency and redundancy of the code are also calculated.

25) **Hamming Code.** Hamming codes are linear block codes with $n=2^m - 1$, $k=2^m - 1 - m$ and minimum distance $d_{min}=3$ and which have a very simple parity check table. The parity check table, which is an $m \times (2^m - 1)$ matrix, has as columns all binary sequences of length m , except the zero sequence.

Create a program for the Hamming code (15,11). For this purpose, use the command `[h,g,n,k] = hamngen()`.

How many bits does the code word have and how many bits does the information message have for this particular code?

What is the parity check table and what is the generator? Does the format of the parity check table agree with the theoretical specifications?

Encode the message

[1 0 0 1 0 1 1 1 1 0]

by writing the encode command: `code=encode(mes,n, k, 'hamming')`. Theoretically verify the resulting codeword.

26) Marcum's Q function. When studying the effect of noise on digital signal transmission, probabilities ($>$) are of interest. The corresponding integral is not calculated analytically and the matrix of the function is used for its calculation.

Marcum's Q. The value of the function $Q(x)$ corresponds to the area of the Gaussian curve from $\bar{y}$ to ∞ .

In Matlab there is the ready-made function `qfunc(x)` which calculates the value of $Q(x)$.

Calculate the values of the function Q (using `qfunc`) for $x=[0, 1, 2, 3, 4, 5, 6, 7]$. Compare them with the corresponding values in the Marcum table of the function Q . Then plot these values and see that Q is a rapidly decreasing function.

27) Noisy signal, SNR. Create a sinusoidal signal of frequency 100Hz, time duration 4sec, width $\sqrt{2}$ and sampling frequency 8KHz. Calculate the power of the sinusoidal signal. $= \frac{1}{N} \sum_{n=1}^N |s[n]|^2$. Then, use the `wgn` function to generate white Gaussian noise with SNR=15dB, (syntax: `wgn(1,N,Pn,'linear')`, where, P_n is the noise power, calculated by the formula $\text{snr}=10\log_{10}(p_x/p_n)$). Add the noise to the signal. Graphically represent the noisy signal.

28) M-adic PSK configuration. Open the bertool (Bit Error Rate Analysis) application of Matlab. Create, with the help of the application, the appropriate graph to show graphically that in M-adic PSK modulation, the noise tolerance decreases as M increases. Use the theoretical analysis, AWGN channel type, modulation type PSK, none channel coding and perfect synchronization. Comment on the graph you created.

29) **Repetition code coding.** A very simple case of block coding is the *simple repetition code*. In this code the codeword consists of the simple repetition of 0 (or 1) n times (n odd). The *decoding* process is based on a majority decision: In the decoding process, an error is observed when at least $(n+1)/2$ of the transmitted bits have been received incorrectly. If the communication channel is a Binary Symmetric Channel, BSC, channel, which exhibits a *probability of error* in the transmitted bit equal to p , then the probability of decoding error for a simple repetition code (n,k) is shown to be given by the following relation,

$$P_e = \sum_{i=(n+1)/2}^n \binom{n}{i} p^i (1-p)^{n-i}$$

Write a program to calculate and represent the *error probability* for a repetition code, given that the error probability for a transmitted bit in the communication channel is equal to $p=0.3$ and to plot the decoding error probability p_e as a function of the *code length* (block length) n . Give n all odd values from 1 to 61. What do you think we pay for the reduction in error probability with increasing code length.

30) **Cyclic code encoding.** Consider a cyclic code with $n=7$ and $k=4$. Generate a generator polynomial using the command `genpoly = cyclpoly(n,k)`. Note that the generated polynomial has its coefficients in ascending order. Next, consider the message `msg=[1 1 1 0]`. Use the command `code=encode(msg, n, k, 'cyclic', genpoly)` to encode the message. What codeword is obtained? Also theoretically encode the message based on the generator polynomial and compare the results.